

MANUAL COMPLETO DE MATEMÁTICAS II

Grado en Ingeniería Informática - 1º Curso

Este documento unifica todos los temas del plan de estudio de Álgebra Lineal y Cálculo Numérico en un único manual para facilitar su lectura, impresión o conversión a formatos como PDF.

Índice General

- **Bloque I: Estructuras y Álgebra Lineal Teórica**
 - [Tema 1: Estructuras Algebraicas y Cuerpos Finitos](#)
 - [Tema 2: Álgebra Matricial y Sistemas de Ecuaciones Lineales](#)
 - [Tema 3: Espacios Vectoriales](#)
 - [Tema 4: Aplicaciones Lineales](#)
 - [Tema 5: Valores y Vectores Propios \(Diagonalización\)](#)
 - [Tema 6: Ortogonalidad](#)
- **Bloque II: Vertiente Numérica y Computacional**
 - [Tema 7: Factorización Matricial y Coste Computacional](#)
 - [Tema 8: Resolución Aproximada de Sistemas \(Métodos Iterativos\)](#)
 - [Tema 9: Laboratorio de Programación Matemática](#)
- **Secciones Finales**
 - [Glosario de Términos](#)
 - [Bibliografía Recomendada](#)

Tema 1: Estructuras Algebraicas y Cuerpos Finitos

El estudio de las estructuras algebraicas abstractas (grupos, anillos y cuerpos) y, especialmente, de los cuerpos finitos (cuerpos de Galois) proporciona el fundamento matemático para la seguridad de la información. En ingeniería informática, estas estructuras sustentan el cifrado de datos (criptografía simétrica y asimétrica), la detección y corrección de errores en la transmisión de datos y el hashing criptográfico.

1. Grupos, Anillos y Cuerpos

Una **estructura algebraica** consiste en un conjunto no vacío dotado de una o más operaciones internas que satisfacen propiedades específicas.

1.1 Grupos

Un conjunto G dotado de una operación binaria interna $*$ (representada como $(G, *)$) es un **grupo** si cumple las siguientes propiedades:

- Asociativa:** $a * (b * c) = (a * b) * c$ para todo $a, b, c \in G$.
- Elemento neutro:** Existe un único elemento $e \in G$ tal que $a * e = e * a = a$ para todo $a \in G$.
- Elemento simétrico (inverso):** Para cada $a \in G$, existe un único $a^{-1} \in G$ tal que $a * a^{-1} = a^{-1} * a = e$.

Grupo abeliano (o conmutativo): Si además cumple la propiedad **conmutativa:** $a * b = b * a$ para todo $a, b \in G$.

1.2 Anillos

Un conjunto A dotado de dos operaciones internas (generalmente suma $+$ y multiplicación $\cdot$), denotado por $(A, +, \cdot)$, es un **anillo** si:

- $(A, +)$ es un grupo abeliano.

2. La multiplicación $\cdot$ es asociativa: $a \cdot (b \cdot c) = (a \cdot b) \cdot c$ para todo $a, b, c \in A$.

3. Se cumple la propiedad **distributiva** del producto respecto a la suma:

$$a \cdot (b + c) = a \cdot b + a \cdot c \quad \text{y} \quad (b + c) \cdot a = b \cdot a + c \cdot a$$

1.3 Cuerpos

Un **cuerpo** (denotado por $\mathbb{K}$) es un anillo conmutativo con elemento unitario ($1 \neq 0$) en el que todo elemento distinto del elemento neutro de la suma (0) tiene un inverso multiplicativo. Es decir:

- $(\mathbb{K}, +)$ es un grupo abeliano (neutro: 0).
- $(\mathbb{K} \setminus \{0\}, \cdot)$ es un grupo abeliano (neutro: 1).

Ejemplos comunes de cuerpos infinitos son los números reales $\mathbb{R}$, los números complejos $\mathbb{C}$ y los racionales $\mathbb{Q}$. Sin embargo, en informática trabajamos mayoritariamente con **cuerpos finitos** (cuerpos con un número finito de elementos).

2. Aritmética Modular y Algoritmo de Euclides Extendido

La aritmética modular opera sobre el conjunto de los restos de división por un entero positivo m , denotado por $Z_m = \{0, 1, 2, \dots, m-1\}$.

Relación de Congruencia

Decimos que dos enteros a y b son **congruentes módulo m** , escrito como:

$$a \equiv b \pmod{m}$$

si su diferencia $a - b$ es un múltiplo entero de m (es decir, m divide a $a - b$).

Propiedades de Z_m

- $(Z_m, +)$ siempre es un grupo abeliano.
- $(Z_m, +, \cdot)$ es un anillo conmutativo unitario.

- $(\mathbb{Z}_m, +, \cdot)$ es un **cuerpo** si y solo si m es un número **primo** p (denotado por $\mathbb{Z}_p$ o $\mathbb{F}_p$). En este caso, cada elemento $a \not\equiv 0 \pmod{p}$ posee un único inverso modular $a^{-1} \pmod{p}$ tal que $a \cdot a^{-1} \equiv 1 \pmod{p}$.

Algoritmo de Euclides Extendido (Cálculo del Inverso Modular)

Para hallar el inverso multiplicativo de a módulo m , requerimos que $\text{mcd}(a, m) = 1$ (es decir, que sean coprimos). El **Algoritmo de Euclides Extendido** calcula enteros x e y (coeficientes de Bézout) tales que:

$$a \cdot x + m \cdot y = \text{mcd}(a, m) = 1$$

Aplicando aritmética modular módulo m :

$$a \cdot x \equiv 1 \pmod{m}$$

Esto significa que x es el inverso modular de a módulo m ($a^{-1} \equiv x \pmod{m}$).

3. Cuerpos Finitos (Cuerpos de Galois $\text{GF}(q)$)

Un cuerpo finito existe si y solo si el número de sus elementos (orden) es de la forma:

$$q = p^m$$

donde p es un número primo (la **característica** del cuerpo) y $m \geq 1$ es un entero positivo. Estos cuerpos se denotan por $\text{GF}(p^m)$.

- Si $m = 1$: El cuerpo es $\text{GF}(p) \cong \mathbb{Z}_p$, con la aritmética modular habitual.
- Si $m > 1$: Los elementos del cuerpo $\text{GF}(p^m)$ no son simplemente enteros modulo p^m . Se representan como **polinomios de grado menor que m** con coeficientes en $\mathbb{Z}_p$. Las operaciones de suma y multiplicación se realizan módulo un **polinomio irreducible** $P(x)$ de grado m .

4. El Toque Informático

4.1 Criptografía Asimétrica (Algoritmo RSA)

El cifrado RSA se basa en la dificultad matemática de factorizar el producto de dos números primos grandes.

1. Se eligen dos primos grandes p y q , y se calcula $n = p \cdot q$.
2. Se calcula la función indicadora de Euler $\phi(n) = (p - 1)(q - 1)$.
3. Se elige un entero coprimo e con $\phi(n)$ ($1 < e < \phi(n)$), que actúa como clave pública.
4. Se calcula el inverso modular $d = e^{-1} \pmod{\phi(n)}$ mediante el algoritmo de Euclides extendido. El valor d es la clave privada.
5. **Cifrado:** Para un mensaje M : $C = M^e \pmod{n}$.
6. **Descifrado:** $M = C^d \pmod{n}$.

4.2 Criptografía Simétrica: Cuerpos de Galois en AES

El estándar de cifrado simétrico **AES** (Advanced Encryption Standard) utiliza el cuerpo finito de Galois $\text{GF}(2^8)$ para realizar operaciones de confusión y difusión de bytes. El polinomio irreducible de grado 8 utilizado en AES es:

$$P(x) = x^8 + x^4 + x^3 + x + 1$$

Cada byte se interpreta como un polinomio de grado menor a 8. La operación de sustitución S-Box de AES consiste en calcular el inverso multiplicativo de cada byte en $\text{GF}(2^8)$ módulo $P(x)$, lo cual proporciona una excelente no-linealidad frente a criptoanálisis lineal.

A continuación, implementamos en Matlab/Octave el algoritmo de Euclides extendido y simulamos el cálculo de claves RSA.

```
% Función que implementa el Algoritmo de Euclides Extendido
function [d, x, y] = euclides_extendido(a, b)
    if b == 0
        d = a;
        x = 1;
        y = 0;
    else
        [d, x1, y1] = euclides_extendido(b, mod(a, b));
        x = y1;
        y = x1 - floor(a / b) * y1;
    end
end
```

```

end
end

% Script de simulación para cálculo de Claves RSA
p = 61; % Número primo
q = 53; % Número primo
n = p * q;
phi = (p - 1) * (q - 1);

e = 17; % Clave pública (debe ser coprimo con phi)

% Calculamos la clave privada d como el inverso modular de e módulo phi
[mcd, d, ~] = euclides_extendido(e, phi);

% El inverso d puede ser negativo, lo ajustamos al rango [0, phi)
d = mod(d, phi);

printf("Parámetros RSA:\n");
printf("Primos: p = %d, q = %d\n", p, q);
printf("Módulo: n = %d\n", n);
printf("Función phi(n) = %d\n", phi);
printf("Clave Pública: e = %d (mcd con phi: %d)\n", e, mcd);
printf("Clave Privada calculada: d = %d\n\n", d);

% Cifrado y descifrado de prueba
mensaje = 65; % Letra 'A' en ASCII
cifrado = mod(mensaje^e, n); % Cifrado básico

% Para el descifrado usamos la función de exponenciación modular rápida para evitar
% overflow
descifrado = 1;
base = mod(cifrado, n);
exp_temp = d;
while exp_temp > 0
    if mod(exp_temp, 2) == 1
        descifrado = mod(descifrado * base, n);
    end
    base = mod(base * base, n);
    exp_temp = floor(exp_temp / 2);
end

printf("Mensaje Original: %d\n", mensaje);
printf("Mensaje Cifrado: %d\n", cifrado);
printf("Mensaje Descifrado: %d\n", descifrado);

```

5. Ejercicios Resueltos

Ejercicio 1

Encontrar el inverso multiplicativo de 13 módulo 60, es decir, resolver $13 \cdot x \equiv 1 \pmod{60}$.

Solución usando el Algoritmo de Euclides Extendido:

1. Fase de divisiones sucesivas (Euclides ordinario):

- $60 = 4 \cdot 13 + 8$
- $13 = 1 \cdot 8 + 5$
- $8 = 1 \cdot 5 + 3$
- $5 = 1 \cdot 3 + 2$
- $3 = 1 \cdot 2 + 1$ (el mcd es 1, confirmando la existencia de inverso).

2. Fase de sustitución hacia atrás (Extended): Despejamos los restos de cada ecuación:

- $1 = 3 - 1 \cdot 2$
- $2 = 5 - 1 \cdot 3$
- $3 = 8 - 1 \cdot 5$
- $5 = 13 - 1 \cdot 8$
- $8 = 60 - 4 \cdot 13$

Ahora, sustituimos secuencialmente:

- $1 = 3 - 1 \cdot (5 - 1 \cdot 3) = 2 \cdot 3 - 1 \cdot 5$
- $1 = 2 \cdot (8 - 1 \cdot 5) - 1 \cdot 5 = 2 \cdot 8 - 3 \cdot 5$
- $1 = 2 \cdot 8 - 3 \cdot (13 - 1 \cdot 8) = 5 \cdot 8 - 3 \cdot 13$
- $1 = 5 \cdot (60 - 4 \cdot 13) - 3 \cdot 13 = 5 \cdot 60 - 23 \cdot 13$

3. Resultado: La ecuación de Bézout obtenida es:

$$(-23) \cdot 13 + 5 \cdot 60 = 1$$

Por tanto, aplicando el módulo 60:

$$-23 \cdot 13 \equiv 1 \pmod{60}$$

Ajustando al rango positivo:

$$x \equiv -23 + 60 = 37 \pmod{60}$$

El inverso multiplicativo de 13 módulo 60 es 37. (Comprobación: $13 \cdot 37 = 481 = 8 \cdot 60 + 1 \equiv 1 \pmod{60}$).

6. Ejercicios Propuestos

1. Resolver la congruencia lineal $7x \equiv 3 \pmod{29}$.
2. Definir las tablas de sumar y multiplicar del cuerpo finito $\text{GF}(3) \cong \mathbb{Z}_3$. ¿Cuáles son los inversos multiplicativos de cada uno de sus elementos no nulos?
3. Explicar por qué el anillo $(\mathbb{Z}_8, +, \cdot)$ no es un cuerpo. Muestra al menos un elemento distinto de cero que carezca de inverso multiplicativo.

Tema 2: Álgebra Matricial y Sistemas de Ecuaciones Lineales

Las matrices son la estructura de datos bidimensional por excelencia en computación. El álgebra matricial permite formular y resolver de manera compacta y eficiente sistemas de ecuaciones lineales gigantescos, los cuales aparecen de forma natural en el renderizado de gráficos 3D, el procesamiento de imágenes, la simulación física de fluidos, la teoría de redes de comunicaciones y la optimización lineal.

1. Operaciones Matriciales y Determinantes

Una **matriz** $A \in M_{m \times n}(K)$ es una disposición rectangular de elementos organizados en m filas y n columnas sobre un cuerpo K (normalmente R o C).

1.1 Operaciones Básicas

- Suma:** Dadas $A, B \in M_{m \times n}(K)$, entonces $(A + B)_{ij} = A_{ij} + B_{ij}$.
- Multiplicación por Escalar:** Para $\lambda \in K$, $(\lambda A)_{ij} = \lambda A_{ij}$.
- Multiplicación de Matrices:** Dadas $A \in M_{m \times p}(K)$ y $B \in M_{p \times n}(K)$, el producto $C = A \cdot B \in M_{m \times n}(K)$ se define como:

$$C_{ij} = \sum_{k=1}^p A_{ik} B_{kj}$$

Nota: La multiplicación de matrices **no es conmutativa** ($A \cdot B \neq B \cdot A$, en general).

1.2 Determinante de una Matriz Cuadrada

El **determinante** de una matriz cuadrada $A \in M_{n \times n}(K)$, denotado por $\det(A)$ o $|A|$, es un escalar que resume importantes propiedades geométricas y algebraicas de la matriz.

- Si $\det(A) \neq 0$: La matriz es **regular** (invertible) y existe la matriz inversa A^{-1} tal que $A \cdot A^{-1} = A^{-1} \cdot A = I_n$.
- Si $\det(A) = 0$: La matriz es **singular** (no invertible).

1.3 Operaciones Elementales de Fila

Son operaciones aplicadas a las filas de una matriz que no alteran el conjunto de soluciones del sistema lineal asociado:

1. Intercambiar dos filas ($F_i \leftrightarrow F_j$).
2. Multiplicar una fila por un escalar no nulo ($F_i \leftarrow \lambda F_i, \lambda \neq 0$).
3. Sumar a una fila otra fila multiplicada por un escalar ($F_i \leftarrow F_i + \lambda F_j$).

2. Resolución de Sistemas de Ecuaciones Lineales (SEL)

Un sistema de m ecuaciones lineales con n incógnitas se escribe matricialmente como:

$$A \cdot x = b$$

donde $A \in M_{m \times n}(\mathbb{K})$ es la **matriz de coeficientes**, $x \in M_{n \times 1}(\mathbb{K})$ es el **vector de incógnitas** y $b \in M_{m \times 1}(\mathbb{K})$ es el **vector de términos independientes**.

2.1 Método de Eliminación de Gauss

Consiste en aplicar operaciones elementales de fila sobre la **matriz ampliada** $(A|b)$ para transformarla en una matriz escalonada superior. Posteriormente, las incógnitas se resuelven mediante **sustitución regresiva**.

El **Método de Gauss-Jordan** lleva el proceso un paso más allá, transformando la matriz A en la identidad (o su forma escalonada reducida por filas), permitiendo leer las soluciones de forma directa.

2.2 Teorema de Rouché-Frobenius (Clasificación de Sistemas)

Denotemos por $\text{rango}(A)$ el número de filas no nulas en su forma escalonada. Sea $(A|b)$ la matriz ampliada del sistema de n incógnitas.

1. **Sistema Compatible (tiene solución):** $\text{rango}(A) = \text{rango}(A|b)$
 - **Determinado (SCD - solución única):** $\text{rango}(A) = n$
 - **Indeterminado (SCI - infinitas soluciones):** $\text{rango}(A) < n$ (el sistema posee $n - \text{rango}(A)$ grados de libertad o parámetros libres).
 2. **Sistema Incompatible (SI - no tiene solución):** $\text{rango}(A) \neq \text{rango}(A|b)$
-

3. El Toque Informático

3.1 Representación de Grafos

En ciencias de la computación, un grafo (como una red de servidores, enlaces web o mapas de transporte) se representa formalmente mediante su **Matriz de Adyacencia** W . Si el grafo tiene n vértices, $W \in M_{n \times n}$ se define como:

$$W_{ij} = \begin{cases} 1 & \text{si hay conexión entre el vértice } i \text{ y el } j \\ 0 & \text{en caso contrario} \end{cases}$$

Una propiedad notable del álgebra matricial es que el término $(W^k)_{ij}$ de la matriz elevada a la potencia k indica exactamente el **número de caminos de longitud k** que existen entre el nodo i y el nodo j .

3.2 Complejidad de la Multiplicación Matricial

La multiplicación matricial ingenua basada en la definición clásica requiere $O(n^3)$ operaciones de punto flotante (FLOPs). En 1969, Volker Strassen diseñó un algoritmo recursivo de tipo "divide y vencerás" que reduce la complejidad a $O(n^{2.807})$ dividiendo las matrices en subbloques y reutilizando combinaciones lineales para reducir el número de multiplicaciones de 8 a 7 en cada nivel recursivo. Hoy en día, las librerías científicas optimizan estas operaciones a nivel de caché del procesador para maximizar el rendimiento.

A continuación, implementamos en Matlab/Octave la reducción escalonada mediante `rref` y la resolución de sistemas usando el operador backslash `\`.

```

% Definición del Sistema de Ecuaciones Lineales
%  $2x + y - z = 8$ 
%  $-3x - y + 2z = -11$ 
%  $-2x + y + 2z = -3$ 

A = [ 2,  1, -1;
      -3, -1,  2;
      -2,  1,  2];

b = [8; -11; -3];

% 1. Comprobación teórica de rangos usando Rouché-Frobenius
rango_A = rank(A);
rango_ampliada = rank([A, b]);
n_incognitas = size(A, 2);

printf("Análisis de Rouché-Frobenius:\n");
printf("Rango(A) = %d\n", rango_A);
printf("Rango(A|b) = %d\n", rango_ampliada);

if rango_A == rango_ampliada
    if rango_A == n_incognitas
        printf("Sistema Compatible Determinado (SCD) - Solución única.\n");
    else
        printf("Sistema Compatible Indeterminado (SCI) - Infinitas soluciones.\n");
    end
else
    printf("Sistema Incompatible (SI) - Sin solución.\n");
end

% 2. Resolución rápida en Matlab/Octave usando el operador de división a izquierda
% (Backslash)
% Este operador utiliza internamente descomposición LU y solver adaptativo
% optimizado.
x_sol = A \ b;

printf("\nSolución del sistema calculada mediante 'A \\ b':\n");
disp(x_sol);

% 3. Inspección detallada del proceso Gauss-Jordan usando rref (Reduced Row Echelon
Form)
M_ampliada = [A, b];
[R, pivotes] = rref(M_ampliada);

printf("Matriz ampliada reducida por filas (Gauss-Jordan):\n");
disp(R);
printf("Nodos pivote identificados en las columnas: ");
disp(pivotes);

```

4. Ejercicios Resueltos

Ejercicio 1

Clasificar y resolver mediante el método de Gauss el siguiente sistema lineal sobre $\mathbb{R}$:

$$\begin{cases} x + 2y - z = 3 \\ 2x + 5y - 4z = 5 \\ 3x + 4y + 2z = 12 \end{cases}$$

Solución:

1. Escribir la matriz ampliada del sistema:

$$(A|b) = \left(\begin{array}{ccc|c} 1 & 2 & -1 & 3 \\ 2 & 5 & -4 & 5 \\ 3 & 4 & 2 & 12 \end{array} \right)$$

2. Hacer ceros debajo del primer pivote ($a_{11} = 1$):

$$\circ F_2 \leftarrow F_2 - 2F_1$$

$$(2 \quad 5 \quad -4 \quad | \quad 5) - (2 \quad 4 \quad -2 \quad | \quad 6) = (0 \quad 1 \quad -2 \quad | \quad -1)$$

$$\circ F_3 \leftarrow F_3 - 3F_1$$

$$(3 \quad 4 \quad 2 \quad | \quad 12) - (3 \quad 6 \quad -3 \quad | \quad 9) = (0 \quad -2 \quad 5 \quad | \quad 3)$$

La matriz ampliada resultante es:

$$\left(\begin{array}{ccc|c} 1 & 2 & -1 & 3 \\ 0 & 1 & -2 & -1 \\ 0 & -2 & 5 & 3 \end{array} \right)$$

3. Hacer ceros debajo del segundo pivote ($a_{22} = 1$):

$$\circ F_3 \leftarrow F_3 + 2F_2$$

$$(0 \quad -2 \quad 5 \quad | \quad 3) + (0 \quad 2 \quad -4 \quad | \quad -2) = (0 \quad 0 \quad 1 \quad | \quad 1)$$

La matriz final escalonada es:

$$\left(\begin{array}{ccc|c} 1 & 2 & -1 & 3 \\ 0 & 1 & -2 & -1 \\ 0 & 0 & 1 & 1 \end{array} \right)$$

4. Clasificación del sistema (Teorema de Rouché-Frobenius):

- El número de filas no nulas es $3 \Rightarrow \text{rango}(A) = \text{rango}(A|b) = 3$.
- Como el número de incógnitas es $n = 3$, el rango es igual a n .
- Por tanto, es un **Sistema Compatible Determinado (SCD)** con solución única.

5. Sustitución regresiva:

- De $F_3: z = 1$
- De $F_2: y - 2z = -1 \Rightarrow y - 2(1) = -1 \Rightarrow y = 1$
- De $F_1: x + 2y - z = 3 \Rightarrow x + 2(1) - 1 = 3 \Rightarrow x + 1 = 3 \Rightarrow x = 2$

La solución única es $x = 2, y = 1, z = 1$.

5. Ejercicios Propuestos

1. Dada la matriz de adyacencia de un grafo de tres nodos:

$$W = \begin{pmatrix} 0 & 1 & 1 \\ 1 & 0 & 0 \\ 1 & 1 & 0 \end{pmatrix}$$

Calcular W^2 y determinar el número de caminos de longitud 2 que existen desde el nodo 3 al nodo 1.

2. Discutir el siguiente sistema de ecuaciones lineales en función del parámetro real k :

$$\begin{cases} x + y + kz = 1 \\ x + ky + z = 1 \\ kx + y + z = 1 \end{cases}$$

3. Demostrar que para dos matrices cualesquiera $A, B \in M_{n \times n}(K)$, se cumple la propiedad distributiva de determinantes: $\det(A \cdot B) = \det(A) \cdot \det(B)$.

Tema 3: Espacios Vectoriales

El concepto de espacio vectorial formaliza la idea de colecciones de objetos (vectores) que se pueden sumar entre sí y multiplicar por números (escalares), preservando propiedades de linealidad. En ingeniería informática, los espacios vectoriales son la base de la representación de datos multidimensionales (embeddings de texto en NLP, características en visión artificial, bases de datos vectoriales) y sustentan la compresión y el procesamiento digital de la información.

1. Definición y Axiomas del Espacio Vectorial

Sea K un cuerpo. Un **espacio vectorial** V sobre K (denotado por $(V, +, \cdot_K)$) es un conjunto no vacío de elementos llamados **vectores**, equipado con dos operaciones:

- Suma de vectores:** $+: V \times V \rightarrow V$
- Multiplicación por un escalar:** $\cdot: K \times V \rightarrow V$

Para que V sea un espacio vectorial, deben cumplirse los 8 axiomas fundamentales:

- Asociatividad de la suma:** $u + (v + w) = (u + v) + w \quad \forall u, v, w \in V$.
- Conmutatividad de la suma:** $u + v = v + u \quad \forall u, v \in V$.
- Elemento neutro de la suma:** Existe $0_V \in V$ tal que $v + 0_V = v \quad \forall v \in V$.
- Elemento opuesto (simétrico):** Para cada $v \in V$, existe $-v \in V$ tal que $v + (-v) = 0_V$.
- Distributividad del escalar:** $\lambda(u + v) = \lambda u + \lambda v \quad \forall \lambda \in K, u, v \in V$.
- Distributividad del vector:** $(\lambda + \mu)v = \lambda v + \mu v \quad \forall \lambda, \mu \in K, v \in V$.
- Compatibilidad de escalares:** $\lambda(\mu v) = (\lambda\mu)v \quad \forall \lambda, \mu \in K, v \in V$.
- Elemento unitario:** $1_K \cdot v = v \quad \forall v \in V$.

Subespacio Vectorial

Un subconjunto no vacío $W \subseteq V$ es un **subespacio vectorial** de V si es en sí mismo un espacio vectorial bajo las operaciones de V .

[!TIP] **Criterio de caracterización de subespacios:** $W \subseteq V$ es un subespacio si y solo si es cerrado bajo combinaciones lineales:

$$\lambda u + \mu v \in W \quad \forall \lambda, \mu \in K, \quad \forall u, v \in W$$

2. Dependencia Lineal, Bases y Dimensión

2.1 Combinación Lineal

Un vector $v \in V$ es una **combinación lineal** de los vectores $v_1, v_2, \dots, v_r \in V$ si existen escalares $\lambda_1, \lambda_2, \dots, \lambda_r \in K$ tales que:

$$v = \lambda_1 v_1 + \lambda_2 v_2 + \dots + \lambda_r v_r$$

El conjunto de todas las combinaciones lineales de un conjunto de vectores se llama **subespacio generado** y se denota por $\text{lin}(v_1, \dots, v_r)$ o $\langle v_1, \dots, v_r \rangle$.

2.2 Independencia Lineal

Decimos que un conjunto de vectores $\{v_1, v_2, \dots, v_r\}$ es **linealmente independiente** (L.I.) si la única combinación lineal que da como resultado el vector nulo 0_V es aquella donde todos los escalares son cero:

$$\lambda_1 v_1 + \lambda_2 v_2 + \dots + \lambda_r v_r = 0_V \implies \lambda_1 = \lambda_2 = \dots = \lambda_r = 0$$

Si existe alguna solución donde no todos los escalares sean cero, los vectores son **linealmente dependientes** (L.D.).

2.3 Bases y Dimensión

Un conjunto de vectores $B = \{e_1, e_2, \dots, e_n\} \subset V$ es una **base** del espacio vectorial V si cumple dos condiciones simultáneamente:

1. Es linealmente independiente.
2. Es un sistema generador de V ($V = \langle e_1, \dots, e_n \rangle$).

La **dimensión** de V , denotada por $\dim(V)$, es el número de vectores que forman cualquiera de sus bases. Todos los vectores de una base dada tienen un único conjunto de coeficientes en K , conocidos como **coordenadas** del vector respecto a dicha base.

3. El Toque Informático

Espacios Vectoriales en Inteligencia Artificial y embeddings

En el campo del Procesamiento del Lenguaje Natural (NLP) y de la Ciencia de Datos, los datos se mapean a vectores en espacios de alta dimensión. Los **embeddings** de palabras (por ejemplo, generados por Word2Vec, GloVe o transformadores modernos de LLMs) asignan a cada palabra un vector en un espacio vectorial continuo (habitualmente $\mathbb{R}^{300}$ o $\mathbb{R}^{768}$). En este espacio:

- Las relaciones semánticas corresponden a operaciones algebraicas. La famosa ecuación:

$$\vec{v}_{\text{rey}} - \vec{v}_{\text{hombre}} + \vec{v}_{\text{mujer}} \approx \vec{v}_{\text{reina}}$$

se realiza en el espacio vectorial $\mathbb{R}^d$.

- La independencia lineal es crucial: si un subconjunto de características de datos (features) es linealmente dependiente, significa que hay redundancia absoluta de datos (multicolinealidad), lo cual desestabiliza los algoritmos de entrenamiento.

A continuación, implementamos en Matlab/Octave un script para verificar si un conjunto de vectores es linealmente independiente y calcular las coordenadas de un vector respecto a una base determinada.

```
% Definimos una base B del espacio vectorial R^3
e1 = [1; 0; 1];
e2 = [1; 1; 0];
e3 = [0; 1; 1];

% Formamos la matriz de la base colocando los vectores como columnas
B = [e1, e2, e3];

% 1. Verificar si B es una base (independencia lineal y dimension 3)
% Si det(B) != 0, los vectores son L.I. y forman una base de R^3
```

```

det_B = det(B);
printf("Determinante de la matriz de vectores: %.4f\n", det_B);

if abs(det_B) > 1e-9
    printf("El conjunto de vectores es Linealmente Independiente (L.I.) y forma una
Base.\n");
else
    printf("Los vectores son Linealmente Dependientes (L.D.). No forman una
base.\n");
end

% 2. Dado un vector v en la base canónica, calcular sus coordenadas en la base B
v = [4; 3; 5];

% Buscamos los coeficientes c = [c1; c2; c3] tales que:
% c1*e1 + c2*e2 + c3*e3 = v ==> B * c = v
coordenadas_c = B \ v;

printf("\nVector v (en base canónica):\n");
disp(v);
printf("Coordenadas del vector v en la base B (c1, c2, c3):\n");
disp(coordenadas_c);

% Verificación de la reconstrucción del vector v
v_reconstruido = B * coordenadas_c;
printf("Vector reconstruido (B * c):\n");
disp(v_reconstruido);

```

4. Ejercicios Resueltos

Ejercicio 1

Determinar si el conjunto de vectores $W = \{(x, y, z) \in \mathbb{R}^3 : 2x - y + z = 0\}$ es un subespacio vectorial de $\mathbb{R}^3$. Si lo es, hallar una base y su dimensión.

Solución:

1. Demostrar que es un subespacio usando el criterio característico:

- **Paso 1:** Comprobar si contiene al vector nulo.

$$2(0) - (0) + (0) = 0 \implies (0, 0, 0) \in W \quad (\text{No es vacío})$$

- **Paso 2:** Tomamos dos vectores arbitrarios de W , $u = (x_1, y_1, z_1)$ y $v = (x_2, y_2, z_2)$, y dos escalares $\lambda, \mu \in \mathbb{R}$. Debemos verificar si $w = \lambda u + \mu v =$

$(\lambda x_1 + \mu x_2, \lambda y_1 + \mu y_2, \lambda z_1 + \mu z_2) \in W$. Evaluamos la condición de pertenencia para w :

$$2(\lambda x_1 + \mu x_2) - (\lambda y_1 + \mu y_2) + (\lambda z_1 + \mu z_2)$$

Reordenamos asociando los coeficientes λ y μ :

$$\lambda(2x_1 - y_1 + z_1) + \mu(2x_2 - y_2 + z_2)$$

Dado que $u, v \in W$, sabemos que $2x_1 - y_1 + z_1 = 0$ y $2x_2 - y_2 + z_2 = 0$. Sustituyendo:

$$\lambda(0) + \mu(0) = 0$$

Por tanto, la combinación lineal pertenece a W , demostrando que W **es un subespacio vectorial** de $\mathbb{R}^3$.

2. **Hallar una base de W** : Despejamos una variable de la ecuación del subespacio, por ejemplo y :

$$y = 2x + z$$

Cualquier vector $(x, y, z) \in W$ puede escribirse sustituyendo y :

$$(x, 2x + z, z) = (x, 2x, 0) + (0, z, z) = x(1, 2, 0) + z(0, 1, 1)$$

Los vectores $v_1 = (1, 2, 0)$ y $v_2 = (0, 1, 1)$ generan W . Como no son proporcionales, son linealmente independientes. Por lo tanto, una base de W es:

$$B_W = \{(1, 2, 0), (0, 1, 1)\}$$

3. **Dimensión**: El número de vectores de la base es 2, por tanto, $\dim(W) = 2$.

5. Ejercicios Propuestos

1. Determinar si los siguientes vectores de $\mathbb{R}^3$ son linealmente independientes o dependientes:

$$u_1 = (1, 1, 1), \quad u_2 = (2, 1, 0), \quad u_3 = (0, 1, 2)$$

2. Sea $P_2[x]$ el espacio de los polinomios de grado menor o igual a 2. Demostrar que el conjunto de polinomios $B = \{1, 1 + x, x + x^2\}$ es una base de $P_2[x]$, y hallar las coordenadas del polinomio $p(x) = 2 - 3x + x^2$ respecto a dicha base.

3. Hallar la dimensión y una base del subespacio intersección $U \cap V$ en $\mathbb{R}^4$, donde:

$$U = \{(x, y, z, t) \in \mathbb{R}^4 : x + y + z = 0\}, \quad V = \{(x, y, z, t) \in \mathbb{R}^4 : z - t = 0\}$$

Tema 4: Aplicaciones Lineales

Las aplicaciones lineales (o transformaciones lineales) describen mapeos entre espacios vectoriales que preservan la estructura de suma de vectores y multiplicación escalar. En ingeniería informática, las aplicaciones lineales son las herramientas fundamentales para manipular coordenadas en gráficos computacionales 3D (motores de renderizado de videojuegos y CAD), procesamiento de señales lineales y transformadas geométricas en visión artificial.

1. Definición y Propiedades de las Aplicaciones Lineales

Sean V y W espacios vectoriales sobre el mismo cuerpo K . Una aplicación $f : V \rightarrow W$ es una **aplicación lineal** (o transformación lineal) si satisface las dos condiciones siguientes para todo $u, v \in V$ y todo $\lambda \in K$:

- Aditividad:** $f(u + v) = f(u) + f(v)$
- Homogeneidad:** $f(\lambda u) = \lambda f(u)$

Estas dos propiedades pueden resumirse en una única condición de preservación de combinaciones lineales:

$$f(\lambda u + \mu v) = \lambda f(u) + \mu f(v) \quad \forall \lambda, \mu \in K, \quad \forall u, v \in V$$

Propiedades inmediatas:

- $f(0_V) = 0_W$
- $f(-u) = -f(u)$

2. Núcleo (Kernel) e Imagen

Asociados a cualquier aplicación lineal $f : V \rightarrow W$, definimos dos subespacios vectoriales críticos que caracterizan el comportamiento geométrico y algebraico de la transformación.

2.1 Núcleo (Kernel)

El **núcleo** de f , denotado por $\ker(f)$ o $\text{Nuc}(f)$, es el conjunto de todos los vectores en el espacio de partida V que se mapean al vector nulo del espacio de llegada W :

$$\ker(f) = \{v \in V : f(v) = 0_W\}$$

- $\ker(f)$ es un subespacio vectorial de V .
- **Monomorfismo (Inyectiva)**: f es inyectiva si y solo si su núcleo contiene únicamente al vector nulo: $\ker(f) = \{0_V\}$.

2.2 Imagen

La **imagen** de f , denotada por $\text{Im}(f)$, es el conjunto de todos los vectores en el espacio de llegada W que provienen de al menos un vector de V :

$$\text{Im}(f) = \{w \in W : \exists v \in V \text{ tal que } f(v) = w\}$$

- $\text{Im}(f)$ es un subespacio vectorial de W .
- **Epimorfismo (Sobreyectiva)**: f es sobreyectiva si y solo si su imagen coincide con todo el espacio de llegada: $\text{Im}(f) = W$.

2.3 Teorema de las Dimensiones (Teorema del Rango-Nulidad)

Si V es de dimensión finita, entonces:

$$\dim(V) = \dim(\ker(f)) + \dim(\text{Im}(f))$$

3. Matriz Asociada y Cambio de Base

Cualquier aplicación lineal entre espacios de dimensión finita puede representarse mediante una multiplicación matricial.

Matriz Asociada

Sean $B_V = \{v_1, \dots, v_n\}$ una base de V y $B_W = \{w_1, \dots, w_m\}$ una base de W . La **matriz asociada a f respecto a las bases B_V y B_W** , denotada por $M(f)_{B_W}^{B_V}$ (o simplemente A), es la matriz de tamaño $m \times n$ cuyas columnas son las coordenadas en B_W de las imágenes de los vectores de la base B_V :

$$A = ([f(v_1)]_{B_W} \quad [f(v_2)]_{B_W} \quad \dots \quad [f(v_n)]_{B_W})$$

El cálculo de la aplicación se reduce a una multiplicación de matriz por vector:

$$[f(v)]_{B_W} = A \cdot [v]_{B_V}$$

Cambio de Base y Semejanza

Si cambiamos de bases en V y W , la nueva matriz asociada A' se calcula mediante la relación:

$$A' = Q^{-1} \cdot A \cdot P$$

donde P es la matriz de cambio de base en el espacio de partida y Q es la matriz de cambio de base en el espacio de llegada.

4. El Toque Informático

Transformaciones y Coordenadas Homogéneas en Gráficos 3D

En gráficos por computadora (como OpenGL o DirectX), los objetos 3D se representan mediante vértices. Para animar u observar la escena desde una cámara, aplicamos transformaciones como traslaciones, rotaciones y escalados.

La rotación y el escalado son transformaciones lineales en $\mathbb{R}^3$, pero la traslación no lo es (ya que $f(v) = v + t$ no cumple $f(0) = 0$). Para unificar todas estas transformaciones en una sola multiplicación matricial que el hardware gráfico (GPU) pueda procesar en paralelo, los informáticos idearon las **Coordenadas Homogéneas**. Se añade una cuarta coordenada ficticia $w = 1$ a cada vector 3D:

$$\vec{p} = \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

Una transformación afín completa (rotación + traslación) se aplica mediante una matriz de transformación de 4×4 :

$$\begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{pmatrix} R_{11} & R_{12} & R_{13} & t_x \\ R_{21} & R_{22} & R_{23} & t_y \\ R_{31} & R_{32} & R_{33} & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

A continuación, implementamos en Matlab/Octave una aplicación lineal de rotación 2D y la aplicamos sobre un conjunto de puntos de un objeto poligonal de forma vectorizada.

```

% 1. Definición del objeto poligonal en 2D (un cuadrado unitario)
% Puntos organizados como columnas: [x; y]
puntos = [0, 1, 1, 0, 0; % Coordenadas X
          0, 0, 1, 1, 0]; % Coordenadas Y

% 2. Matriz de la transformación lineal de Rotación de ángulo theta
theta = pi / 4; % Rotación de 45 grados (en radianes)
R = [cos(theta), -sin(theta);
     sin(theta),  cos(theta)];

% 3. Aplicación de la transformación de forma vectorizada (multiplicación
matricial)
puntos_rotados = R * puntos;

% Visualización gráfica
figure;
plot(puntos(1, :), puntos(2, :), 'b-o', 'LineWidth', 2, 'DisplayName', 'Original');
hold on;
plot(puntos_rotados(1, :), puntos_rotados(2, :), 'r-s', 'LineWidth', 2,
     'DisplayName', 'Rotado 45°');
grid on;
axis equal;
xlabel('Eje X');
ylabel('Eje Y');
title('Transformación Lineal: Rotación 2D');
legend('show');
hold off;

% Imprimir las coordenadas transformadas
printf("Coordenadas Originales:\n");
disp(puntos);
printf("Coordenadas Rotadas:\n");
disp(puntos_rotados);

```

5. Ejercicios Resueltos

Ejercicio 1

Sea la aplicación lineal $f : \mathbb{R}^3 \rightarrow \mathbb{R}^2$ definida por:

$$f(x, y, z) = (x - 2y, 2x - 4y + z)$$

1. Hallar la matriz asociada a f respecto a las bases canónicas.
2. Calcular una base y la dimensión de su núcleo $\ker(f)$.
3. Determinar la dimensión de su imagen $\text{Im}(f)$.

Solución:

1. **Matriz asociada respecto a bases canónicas C_3 y C_2 :** Calculamos las imágenes de los vectores de la base canónica de $\mathbb{R}^3$:

- $f(1, 0, 0) = (1 - 2(0), 2(1) - 4(0) + 0) = (1, 2)$
- $f(0, 1, 0) = (0 - 2(1), 2(0) - 4(1) + 0) = (-2, -4)$
- $f(0, 0, 1) = (0 - 2(0), 2(0) - 4(0) + 1) = (0, 1)$

Colocando estas imágenes como columnas de la matriz A :

$$A = \begin{pmatrix} 1 & -2 & 0 \\ 2 & -4 & 1 \end{pmatrix}$$

2. **Calcular una base y dimensión del núcleo:** El núcleo es el conjunto de vectores (x, y, z) tales que $f(x, y, z) = (0, 0)$:

$$\begin{cases} x - 2y = 0 \\ 2x - 4y + z = 0 \end{cases}$$

Resolvemos el sistema. De la primera ecuación: $x = 2y$. Sustituyendo en la segunda: $2(2y) - 4y + z = 0 \Rightarrow 4y - 4y + z = 0 \Rightarrow z = 0$. Las soluciones son de la forma $(2y, y, 0) = y(2, 1, 0)$. Por tanto, una base del núcleo es:

$$B_{\ker(f)} = \{(2, 1, 0)\}$$

La dimensión del núcleo es $\dim(\ker(f)) = 1$.

3. **Calcular dimensión de la imagen:** Aplicamos el Teorema de las Dimensiones:

$$\dim(\mathbb{R}^3) = \dim(\ker(f)) + \dim(\text{Im}(f)) \Rightarrow 3 = 1 + \dim(\text{Im}(f)) \Rightarrow \dim(\text{Im}(f)) = 2$$

(Nota: Como la dimensión de la imagen es igual a la dimensión del espacio de llegada $\mathbb{R}^2$, la aplicación es sobreyectiva).

6. Ejercicios Propuestos

1. Demostrar si la aplicación $g : \mathbb{R}^2 \rightarrow \mathbb{R}^2$ dada por $g(x, y) = (x^2, x + y)$ es lineal o no lo es.
2. Dada la aplicación lineal de proyección $P : \mathbb{R}^3 \rightarrow \mathbb{R}^3$ con matriz asociada respecto a la base canónica:

$$A = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{pmatrix}$$

Determinar su núcleo e imagen, y describir geoméricamente el efecto físico de esta aplicación en el espacio tridimensional.

3. Sea $f : \mathbb{R}^2 \rightarrow \mathbb{R}^2$ una aplicación lineal cuya matriz en la base canónica es $A = \begin{pmatrix} 1 & 2 \\ 2 & 1 \end{pmatrix}$. Calcular la matriz asociada a f en la nueva base $B = \{(1, 1), (1, -1)\}$.

Tema 5: Valores y Vectores Propios (Diagonalización)

El cálculo de autovalores y autovectores permite simplificar representaciones matriciales complejas, revelando direcciones preferentes de variación espacial y comportamiento dinámico. En ingeniería informática, la diagonalización es el núcleo de la compresión de datos y reducción de ruido (PCA), de los motores de búsqueda modernos (PageRank de Google), de la clasificación automática y del análisis de sistemas dinámicos recursivos.

1. Definición y Polinomio Característico

Sea $A \in M_{n \times n}(K)$ una matriz cuadrada sobre un cuerpo K .

Autovalores y Autovectores

Decimos que un escalar $\lambda \in K$ es un **valor propio** (autovalor o eigenvalue) de A si existe un vector no nulo $v \in M_{n \times 1}(K)$ ($v \neq 0$) tal que:

$$A \cdot v = \lambda v$$

El vector v se denomina **vector propio** (autovector o eigenvector) asociado al autovalor λ .

Geométricamente, multiplicar un autovector por la matriz A no cambia su dirección en el espacio; únicamente altera su longitud por el factor de escala λ .

Polinomio Característico

Para hallar los autovalores de A , reescribimos la ecuación como:

$$(A - \lambda I_n)v = 0$$

Dado que buscamos soluciones no nulas ($v \neq 0$), el sistema homogéneo debe ser compatible indeterminado, lo cual ocurre si y solo si el determinante de la matriz de

coeficientes es nulo. Esto define la **ecuación característica**:

$$p(\lambda) = \det(A - \lambda I_n) = 0$$

El determinante $p(\lambda)$ es un polinomio de grado n en λ , conocido como **polinomio característico**. Las raíces de este polinomio son precisamente los autovalores de la matriz.

2. Multiplicidades y Condiciones de Diagonalización

Multiplicidad Algebraica (m_{alg})

Es la multiplicidad de λ_i como raíz del polinomio característico.

Multiplicidad Geométrica (m_{geom})

Es la dimensión del subespacio propio asociado a λ_i , denotado por V_{λ_i} (que contiene todos los autovectores asociados a λ_i junto con el vector nulo):

$$V_{\lambda_i} = \ker(A - \lambda_i I_n)$$

$$m_{\text{geom}}(\lambda_i) = \dim(V_{\lambda_i}) = n - \text{rango}(A - \lambda_i I_n)$$

[!WARNING] Para cualquier autovalor λ , siempre se cumple que: $1 \leq m_{\text{geom}}(\lambda) \leq m_{\text{alg}}(\lambda)$

2.3 Diagonalizabilidad de una Matriz

Decimos que una matriz cuadrada $A \in M_{n \times n}(\mathbb{K})$ es **diagonalizable** si existe una matriz invertible P (matriz de paso) y una matriz diagonal D tales que:

$$P^{-1} \cdot A \cdot P = D \quad \Rightarrow \quad A = P \cdot D \cdot P^{-1}$$

Teorema Fundamental de Diagonalización

Una matriz A de tamaño $n \times n$ es diagonalizable en $\mathbb{K}$ si y solo si:

1. El polinomio característico se descompone completamente en factores lineales sobre $\mathbb{K}$ (todas sus raíces pertenecen a $\mathbb{K}$).
2. Para cada autovalor λ_i , su multiplicidad geométrica es igual a su multiplicidad algebraica:

$$m_{\text{geom}}(\lambda_i) = m_{\text{alg}}(\lambda_i) \quad \forall i$$

En este caso:

- La matriz diagonal D contiene los autovalores en su diagonal principal.
- La matriz P contiene como columnas a los autovectores asociados correspondientes en el mismo orden.

3. El Toque Informático

3.1 El Algoritmo PageRank de Google

En los inicios de Google, Larry Page y Sergey Brin diseñaron el algoritmo PageRank para ordenar las páginas de los resultados de búsqueda. Internet se modela como un grafo dirigido de enlaces:

- Si una página j enlaza a la página i , se le transfiere una parte de su importancia.
- Definimos la **Matriz de Transición de Google** G , que es estocástica (la suma de sus columnas es 1).
- La importancia (PageRank) de todas las páginas de la red corresponde al vector de estado estacionario v que cumple:

$$G \cdot v = 1 \cdot v$$

Es decir, el PageRank es el **vector propio asociado al autovalor dominante** $\lambda = 1$ de la matriz G . Se calcula de forma masiva e iterativa mediante el **Método de las Potencias**.

3.2 Reducción de Dimensionalidad (PCA)

En machine learning, cuando los datos tienen cientos de variables, el **Análisis de Componentes Principales** (PCA) los proyecta a un espacio de menor dimensión. Las direcciones en las que los datos varían más corresponden a los **autovectores** de la **Matriz de Covarianza** de los datos, ordenados de mayor a menor según sus autovalores correspondientes (varianza explicada).

A continuación, implementamos en Matlab/Octave el cálculo de autovalores, autovectores y reconstrucción de la diagonalización.

```
% Definimos la matriz A
A = [ 4, -1, 6;
      2,  1, 6;
      2, -1, 8];

% 1. Calcular autovalores y autovectores usando la función 'eig'
% D es la matriz diagonal de autovalores
% P es la matriz cuyas columnas son los autovectores
[P, D] = eig(A);

printf("Matriz de Autovectores P (columnas):\n");
disp(P);
printf("Matriz Diagonal de Autovalores D:\n");
disp(D);

% 2. Verificar la igualdad teórica A * P = P * D
LHS = A * P;
RHS = P * D;

printf("Verificación A * P (Lado Izquierdo):\n");
disp(LHS);
printf("Verificación P * D (Lado Derecho):\n");
disp(RHS);

% Comprobar numéricamente la diferencia para tolerar errores de redondeo de coma
flotante
error_max = max(max(abs(LHS - RHS)));
printf("Diferencia máxima absoluta: %.2e\n", error_max);

% 3. Reconstruir la matriz original A = P * D * inv(P)
A_reconstruida = P * D * inv(P);
printf("\nMatriz A reconstruida (P * D * P^-1):\n");
disp(A_reconstruida);
```

4. Ejercicios Resueltos

Ejercicio 1

Determinar si la matriz A es diagonalizable sobre $\mathbb{R}$:

$$A = \begin{pmatrix} 3 & 0 \\ 1 & 3 \end{pmatrix}$$

Solución:

1. **Calcular el polinomio característico:**

$$p(\lambda) = \det(A - \lambda I_2) = \det \begin{pmatrix} 3-\lambda & 0 \\ 1 & 3-\lambda \end{pmatrix} = (3-\lambda)^2 = 0$$

La única raíz es $\lambda = 3$ con **multiplicidad algebraica** $m_{\text{alg}}(3) = 2$.

2. **Calcular la multiplicidad geométrica** $m_{\text{geom}}(3)$:

$$m_{\text{geom}}(3) = 2 - \text{rango}(A - 3I_2)$$

Calculamos $A - 3I_2$:

$$A - 3I_2 = \begin{pmatrix} 3-3 & 0 \\ 1 & 3-3 \end{pmatrix} = \begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix}$$

El rango de esta matriz es claramente 1 (tiene una fila no nula). Por tanto:

$$m_{\text{geom}}(3) = 2 - 1 = 1$$

3. **Conclusión:** Dado que:

$$m_{\text{geom}}(3) = 1 \neq m_{\text{alg}}(3) = 2$$

No se cumple la igualdad de multiplicidades. Por lo tanto, la matriz A **no es diagonalizable** en $\mathbb{R}$.

5. Ejercicios Propuestos

1. Dada la matriz $A = \begin{pmatrix} 1 & 2 \\ 2 & 1 \end{pmatrix}$, hallar sus autovalores y sus autovectores asociados, y construir las matrices de diagonalización D y P .
2. Demostrar que una matriz A de tamaño $n \times n$ y su traspuesta A^T poseen exactamente los mismos autovalores. (*Pista: Compara sus polinomios característicos y las propiedades del determinante de la traspuesta*).

3. Explicar cómo se comporta la potencia k -ésima de una matriz diagonalizable A^k utilizando su descomposición diagonal $P \cdot D \cdot P^{-1}$. ¿Cómo simplifica esto el cálculo algorítmico frente a la multiplicación directa repetida?

Tema 6: Ortogonalidad

La ortogonalidad extiende el concepto geométrico de perpendicularidad a espacios abstractos de dimensión general. En ingeniería informática, los conceptos de ortogonalidad sustentan la regresión por mínimos cuadrados para ajustar curvas a datos experimentales (Machine Learning), la compresión de señales digitales (transformadas de Fourier y coseno discretas) y las métricas de distancia en motores de búsqueda (similitud de coseno).

1. Producto Escalar, Norma y Distancia

Sea V un espacio vectorial sobre el cuerpo $\mathbb{R}$.

Producto Escalar

Un **producto escalar** en V es una aplicación que asocia a cada par de vectores $u, v \in V$ un número real, denotado por $\langle u, v \rangle$ (o $u \cdot v$), que satisface las siguientes propiedades para todo $u, v, w \in V$ y $\lambda \in \mathbb{R}$:

1. **Simetría:** $\langle u, v \rangle = \langle v, u \rangle$
2. **Linealidad en la primera componente:** $\langle \lambda u + \mu v, w \rangle = \lambda \langle u, w \rangle + \mu \langle v, w \rangle$
3. **Definida positiva:** $\langle v, v \rangle \geq 0$, y $\langle v, v \rangle = 0 \iff v = 0_V$.

Un espacio vectorial real dotado de un producto escalar se denomina **espacio euclídeo**.

Norma y Distancia

- Definimos la **norma** (longitud) de un vector como:

$$\|v\| = \sqrt{\langle v, v \rangle}$$

- Definimos la **distancia** entre dos vectores como:

$$d(u, v) = \|u - v\|$$

- Definimos el **ángulo** θ entre dos vectores no nulos a partir de la relación:

$$\cos \theta = \frac{\langle u, v \rangle}{\|u\| \cdot \|v\|}$$

2. Ortogonalidad y Proyección Ortogonal

Dos vectores u y v son **ortogonales** (perpendiculares) si su producto escalar es nulo:

$$u \perp v \Leftrightarrow \langle u, v \rangle = 0$$

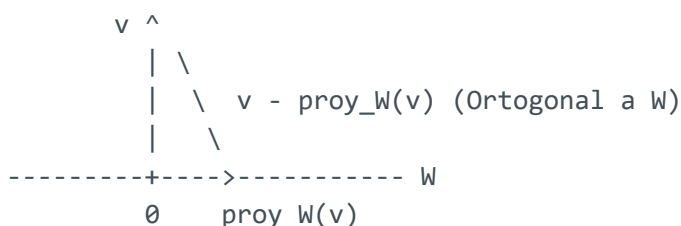
Un conjunto de vectores $S = \{v_1, \dots, v_r\}$ es **ortogonal** si todos sus vectores son mutuamente ortogonales ($v_i \cdot v_j = 0$ para todo $i \neq j$). Si además cada vector tiene norma unitaria ($\|v_i\| = 1$), el conjunto es **ortonormal**.

Proyección Ortogonal sobre un Subespacio

Sea W un subespacio de V con una base ortonormal $\{u_1, u_2, \dots, u_k\}$. Para cualquier vector $v \in V$, definimos su **proyección ortogonal sobre W** , denotada por $\text{proy}_W(v)$, como:

$$\text{proy}_W(v) = \langle v, u_1 \rangle u_1 + \langle v, u_2 \rangle u_2 + \dots + \langle v, u_k \rangle u_k$$

Geométricamente, $\text{proy}_W(v)$ es el vector en W que está a la **mínima distancia** de v .



3. Proceso de Ortogonalización de Gram-Schmidt

Dado un conjunto de vectores linealmente independientes $\{v_1, v_2, \dots, v_n\}$ que forma una base del espacio V , el **Método de Gram-Schmidt** construye de forma constructiva una base ortogonal $\{u_1, u_2, \dots, u_n\}$ del mismo espacio:

1.
$$u_1 = v_1$$

2.
$$u_2 = v_2 - \frac{\langle v_2, u_1 \rangle}{\langle u_1, u_1 \rangle} u_1$$

3.
$$u_3 = v_3 - \frac{\langle v_3, u_1 \rangle}{\langle u_1, u_1 \rangle} u_1 - \frac{\langle v_3, u_2 \rangle}{\langle u_2, u_2 \rangle} u_2$$

4. En general, para el paso k :

$$u_k = v_k - \sum_{i=1}^{k-1} \frac{\langle v_k, u_i \rangle}{\langle u_i, u_i \rangle} u_i$$

Para obtener una base **ortonormal** $\{q_1, \dots, q_n\}$, normalizamos cada uno de los vectores resultantes:

$$q_i = \frac{u_i}{\|u_i\|}$$

4. El Toque Informático

4.1 Motores de Recomendación (Métrica de Similitud de Coseno)

En sistemas de filtrado colaborativo (como los de Netflix o Amazon) y en recuperación de información, comparamos perfiles de usuarios o documentos. Estos perfiles se representan como vectores de características de alta dimensión. Para medir la similitud semántica sin importar el tamaño del documento o el número total de interacciones del usuario, calculamos el **coseno del ángulo entre los vectores** (Similitud de Coseno):

$$\text{Similitud}(A, B) = \cos \theta = \frac{A \cdot B}{\|A\| \cdot \|B\|}$$

- Si $\cos \theta = 1$: Los perfiles son idénticos en dirección (misma proporción de gustos).
- Si $\cos \theta = 0$: Los perfiles son ortogonales (no comparten intereses).

4.2 Ajuste por Mínimos Cuadrados (Regresión Lineal)

Cuando queremos ajustar un modelo predictivo lineal $y = X\beta$ a datos ruidosos, el sistema de ecuaciones suele ser incompatible (más ecuaciones que incógnitas). La mejor aproximación en sentido geométrico consiste en proyectar ortogonalmente el vector de datos observados y sobre el subespacio imagen de la matriz X . Esto se resuelve analíticamente mediante las **Ecuaciones Normales**:

$$X^T X \beta = X^T y \Rightarrow \beta = (X^T X)^{-1} X^T y$$

A continuación, implementamos en Matlab/Octave el método de Gram-Schmidt para construir una base ortonormal.

```
% Definimos una base original V de R^3
V = [1, 1, 0;
     1, 0, 1;
     0, 1, 1]; % Cada columna es un vector v_i

n = size(V, 2);
U = zeros(size(V)); % Matriz para guardar la base ortogonal

% Algoritmo de Gram-Schmidt
for k = 1:n
    U(:, k) = V(:, k);
    for j = 1:(k-1)
        % Calculamos la proyección y la restamos
        proyeccion = (U(:, j)' * V(:, k)) / (U(:, j)' * U(:, j));
        U(:, k) = U(:, k) - proyeccion * U(:, j);
    end
end

% Normalización para obtener base ortonormal Q
Q = zeros(size(U));
for k = 1:n
    Q(:, k) = U(:, k) / norm(U(:, k));
end

printf("Base Original V:\n");
disp(V);
printf("Base Ortogonal calculada U:\n");
disp(U);
printf("Base Ortonormal final Q:\n");
```

```
disp(Q);

% Verificación de la ortonormalidad: Q^T * Q debe ser igual a la Identidad
identidad_aprox = Q' * Q;
printf("Verificación Q^T * Q:\n");
disp(identidad_aprox);
```

5. Ejercicios Resueltos

Ejercicio 1

Aplicar el método de Gram-Schmidt para ortogonalizar la base de $\mathbb{R}^2$ dada por los vectores $v_1 = (1, 1)$ y $v_2 = (1, 2)$ utilizando el producto escalar usual.

Solución:

1. Calcular el primer vector ortogonal u_1 :

$$u_1 = v_1 = (1, 1)$$

2. Calcular el segundo vector ortogonal u_2 :

$$u_2 = v_2 - \frac{\langle v_2, u_1 \rangle}{\langle u_1, u_1 \rangle} u_1$$

Calculamos los productos escalares:

- $\langle v_2, u_1 \rangle = (1, 2) \cdot (1, 1) = 1 \cdot 1 + 2 \cdot 1 = 3$
- $\langle u_1, u_1 \rangle = (1, 1) \cdot (1, 1) = 1^2 + 1^2 = 2$ Sustituimos en la fórmula:

$$u_2 = (1, 2) - \frac{3}{2}(1, 1) = (1, 2) - \left(\frac{3}{2}, \frac{3}{2}\right) = \left(-\frac{1}{2}, \frac{1}{2}\right)$$

3. Comprobación:

$$u_1 \cdot u_2 = (1, 1) \cdot \left(-\frac{1}{2}, \frac{1}{2}\right) = -\frac{1}{2} + \frac{1}{2} = 0 \quad (\text{Correcto, son ortogonales})$$

4. Normalización para base ortonormal:

- $\|u_1\| = \sqrt{2} \Rightarrow q_1 = \left(\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}}\right)$
- $\|u_2\| = \sqrt{\left(-\frac{1}{2}\right)^2 + \left(\frac{1}{2}\right)^2} = \sqrt{\frac{1}{4} + \frac{1}{4}} = \sqrt{\frac{1}{2}} = \frac{1}{\sqrt{2}}$

$$q_2 = \frac{u_2}{\|u_2\|} = \sqrt{2} \left(-\frac{1}{2}, \frac{1}{2}\right) = \left(-\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}}\right)$$

La base ortogonal es $\{(1, 1), (-1/2, 1/2)\}$ y la base ortonormal es $\{(1/\sqrt{2}, 1/\sqrt{2}), (-1/\sqrt{2}, 1/\sqrt{2})\}$.

6. Ejercicios Propuestos

1. Dados los vectores de características en un motor de búsqueda:

$$A = (1, 2, 0, 1), \quad B = (2, 1, 1, 0)$$

Calcular su similitud de coseno y explicar cuantitativamente qué tan similares son semánticamente.

2. Hallar la proyección ortogonal del vector $v = (1, 5)$ sobre la recta (subespacio) generada por el vector $u = (3, 4)$ en $\mathbb{R}^2$.
3. Demostrar algebraicamente que si un conjunto de vectores no nulos $\{v_1, \dots, v_k\}$ es ortogonal, entonces es linealmente independiente. (*Pista: Plantea la ecuación de combinación lineal a cero y multiplica escalarmente por cada vector del conjunto*).

Tema 7: Factorización Matricial y Coste Computacional

La resolución directa de sistemas de ecuaciones lineales mediante el método de Gauss es computacionalmente costosa y puede verse afectada por inestabilidad numérica. Las factorizaciones matriciales descomponen una matriz en un producto de matrices más sencillas (triangulares, ortogonales, simétricas) que facilitan cálculos repetidos. En la ingeniería informática, comprender estas descomposiciones es crucial para optimizar algoritmos científicos, simulaciones físicas de alto rendimiento y análisis estructural de redes.

1. Descomposición LU (con y sin pivotaje)

La **factorización LU** expresa una matriz cuadrada $A \in M_{n \times n}(\mathbb{R})$ como el producto de una matriz triangular inferior con unos en la diagonal L (Lower) y una matriz triangular superior U (Upper):

$$A = L \cdot U$$

$$A = \begin{bmatrix} * & * & * \\ * & * & * \\ * & * & * \end{bmatrix} \quad L = \begin{bmatrix} 1 & 0 & 0 \\ * & 1 & 0 \\ * & * & 1 \end{bmatrix} \quad U = \begin{bmatrix} * & * & * \\ 0 & * & * \\ 0 & 0 & * \end{bmatrix}$$

Una vez que tenemos la descomposición $L \cdot U$, resolver el sistema $A \cdot x = b$ se reduce a resolver dos sistemas triangulares sucesivos sumamente rápidos ($O(n^2)$):

- Resolver $L \cdot y = b$ mediante **sustitución directa**.
- Resolver $U \cdot x = y$ mediante **sustitución regresiva**.

Estrategia de Pivotaje Parcial (Factorización PLU)

Si durante la eliminación de Gauss un elemento de la diagonal (pivote) es cero o cercano a cero, la división matemática introduce graves errores de redondeo. Para resolver esto se aplica **pivotaje parcial**: intercambiamos filas para colocar el elemento de mayor valor absoluto de la columna en la diagonal. Matricialmente, esto se representa como:

$$P \cdot A = L \cdot U$$

donde P es una **matriz de permutación** (matriz identidad con sus filas intercambiadas).

2. Factorización de Cholesky

Si la matriz A es **simétrica** ($A = A^T$) y **definida positiva** ($x^T A x > 0$ para todo vector no nulo x), se puede descomponer de forma más eficiente que LU como el producto de una matriz triangular inferior L por su propia traspuesta:

$$A = L \cdot L^T$$

Ventaja Computacional

- Requiere únicamente la mitad de operaciones en punto flotante que la descomposición LU estándar.
 - Es numéricamente muy estable y no requiere pivotaje.
-

3. Factorización QR

La **factorización QR** descompone una matriz $A \in M_{m \times n}(\mathbb{R})$ ($m \geq n$) en el producto de una matriz ortogonal Q ($Q^T Q = I$) y una matriz triangular superior R :

$$A = Q \cdot R$$

Se implementa mediante tres enfoques matemáticos principales:

1. **Ortogonalización de Gram-Schmidt** (puede sufrir inestabilidad por errores de redondeo).

- 2. **Transformaciones de Householder** (reflexiones ortogonales, muy estable).
- 3. **Rotaciones de Givens** (rotaciones locales, ideal para matrices dispersas).

4. Coste Computacional y Análisis Algorítmico

En informática, la eficiencia se mide por el número de **FLOPs** (operaciones de punto flotante: sumas, restas, productos y divisiones). A continuación, se detalla el coste de las principales operaciones para matrices de tamaño $n \times n$:

Operación / Factorización	Coste Computacional (FLOPs)	Complejidad Temporal
Multiplicación clásica $A \cdot B$	$2n^3$	$O(n^3)$
Cálculo de la Inversa A^{-1}	$2n^3$	$O(n^3)$
Descomposición LU / Eliminación de Gauss	$\frac{2}{3}n^3$	$O(n^3)$
Descomposición de Cholesky	$\frac{1}{3}n^3$	$O(n^3)$
Descomposición QR (Householder)	$\frac{4}{3}n^3$	$O(n^3)$
Sustitución directa/regresiva (triangular)	n^2	$O(n^2)$

[!IMPORTANT] **Por qué no calcular la inversa (A^{-1}):** Resolver un sistema mediante $x = A^{-1}b$ es computacionalmente ineficiente y numéricamente inestable en comparación con las factorizaciones. Si tenemos múltiples vectores de carga b (como en animaciones de física frame a frame), realizamos la factorización **una sola vez** ($O(n^3)$) y en cada frame resolvemos mediante sustituciones rápidas ($O(n^2)$), en lugar de volver a invertir la matriz.

A continuación, implementamos en Matlab/Octave las descomposiciones LU y Cholesky.

```
% Definimos una matriz simétrica y definida positiva A
A = [ 4, 12, -16;
```

```

    12, 37, -43;
    -16, -43, 98];

b = [24; 86; -182];

% 1. Descomposición LU con pivotaje parcial (P * A = L * U)
[L_lu, U_lu, P_lu] = lu(A);

printf("Matriz de Permutación P:\n");
disp(P_lu);
printf("Matriz Triangular Inferior L (LU):\n");
disp(L_lu);
printf("Matriz Triangular Superior U (LU):\n");
disp(U_lu);

% 2. Descomposición de Cholesky (A = L_chol * L_chol')
% Nota: En Matlab/Octave, 'chol' devuelve por defecto la triangular superior U (A = U'*U)
% Para obtener la inferior L, pasamos el parámetro 'lower'.
L_chol = chol(A, 'lower');

printf("Matriz de Cholesky L (triangular inferior):\n");
disp(L_chol);

% Verificar que L * L^T = A
printf("Verificación L * L^T:\n");
disp(L_chol * L_chol');

% 3. Resolución del sistema usando Cholesky
% L * y = b ==> y = L \ b (Sustitución directa)
% L' * x = y ==> x = L' \ y (Sustitución regresiva)
y = L_chol \ b;
x_sol = L_chol' \ y;

printf("Solución del sistema calculada mediante Cholesky:\n");
disp(x_sol);

```

5. Ejercicios Resueltos

Ejercicio 1

Calcular de forma manual la descomposición LU (sin pivotaje) de la siguiente matriz:

$$A = \begin{pmatrix} 2 & 1 \\ 4 & 7 \end{pmatrix}$$

Solución:

1. **Hacer la eliminación de Gauss sobre A para hallar U :** Queremos eliminar el elemento $a_{21} = 4$. Para ello multiplicamos la fila 1 por el multiplicador $m_{21} = \frac{a_{21}}{a_{11}} = \frac{4}{2} = 2$.

◦ $F_2 \leftarrow F_2 - m_{21}F_1 = F_2 - 2F_1$

$$(4 \quad 7) - 2(2 \quad 1) = (4 \quad 7) - (4 \quad 2) = (0 \quad 5)$$

La matriz resultante escalonada superior es U :

$$U = \begin{pmatrix} 2 & 1 \\ 0 & 5 \end{pmatrix}$$

2. **Construir la matriz triangular inferior L :** La matriz L tiene 1s en la diagonal y los multiplicadores en la parte inferior:

$$L = \begin{pmatrix} 1 & 0 \\ m_{21} & 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 2 & 1 \end{pmatrix}$$

3. **Verificación del producto $L \cdot U$:**

$$L \cdot U = \begin{pmatrix} 1 & 0 \\ 2 & 1 \end{pmatrix} \begin{pmatrix} 2 & 1 \\ 0 & 5 \end{pmatrix} = \begin{pmatrix} 1 \cdot 2 + 0 \cdot 0 & 1 \cdot 1 + 0 \cdot 5 \\ 2 \cdot 2 + 1 \cdot 0 & 2 \cdot 1 + 1 \cdot 5 \end{pmatrix} = \begin{pmatrix} 2 & 1 \\ 4 & 7 \end{pmatrix} = A \quad (\text{Correcto})$$

6. Ejercicios Propuestos

1. Dada la matriz $A = \begin{pmatrix} 4 & 2 \\ 2 & 10 \end{pmatrix}$, hallar de forma manual su descomposición de Cholesky $A = L \cdot L^T$.
2. Describir el efecto de multiplicar una matriz A por una matriz de permutación P a la izquierda ($P \cdot A$) frente a multiplicarla por la derecha ($A \cdot P$).
3. Sea un sistema físico modelado por $Ax = b$. Si A es una matriz de 1000×1000 y necesitamos resolver el sistema para 50 vectores b distintos:
 - Calcular el coste aproximado en operaciones de punto flotante (FLOPs) si calculamos A^{-1} y multiplicamos $x = A^{-1}b$.
 - Calcular el coste si calculamos la factorización LU y resolvemos mediante sustituciones.
 - Comparar ambos enfoques y determinar cuál es más eficiente.

Tema 8: Resolución Aproximada de Sistemas (Métodos Iterativos)

Cuando tratamos con sistemas de ecuaciones lineales gigantescos (millones de incógnitas), habituales en computación científica, procesamiento de imágenes o análisis de redes sociales, los métodos directos (como Gauss o LU) son inviables debido a su coste computacional $O(n^3)$ y al almacenamiento de memoria. Los **métodos iterativos** resuelven esta limitación aproximando la solución de manera progresiva mediante sucesivas iteraciones más sencillas, siendo idóneos para **matrices dispersas** (matrices donde la inmensa mayoría de sus elementos son ceros).

1. Concepto General de los Métodos Iterativos

Para resolver $A \cdot x = b$, dividimos (descomponemos) la matriz de coeficientes A como:

$$A = M - N$$

donde M es una matriz fácilmente invertible. Reescribimos el sistema como:

$$(M - N)x = b \Rightarrow Mx = Nx + b \Rightarrow x = M^{-1}Nx + M^{-1}b$$

Esto define la **ecuación iterativa** de recurrencia:

$$x^{(k+1)} = B \cdot x^{(k)} + c \quad (\text{para } k = 0, 1, 2, \dots)$$

donde:

- $B = M^{-1}N$ es la **matriz de iteración**.
 - $c = M^{-1}b$ es un vector constante.
 - $x^{(0)}$ es una aproximación inicial arbitraria.
-

2. Los Algoritmos de Jacobi y Gauss-Seidel

Descomponemos la matriz A en la suma de sus componentes:

$$A = D - L - U$$

donde D es la diagonal de A , $-L$ es la parte triangular inferior estricta y $-U$ es la parte triangular superior estricta.

$$A = \begin{bmatrix} D \\ \end{bmatrix} - \begin{bmatrix} L \\ \end{bmatrix} - \begin{bmatrix} U \\ \end{bmatrix}$$

2.1 Método de Jacobi

El método de Jacobi calcula la nueva aproximación $x^{(k+1)}$ usando únicamente los valores de la iteración anterior $x^{(k)}$. Matricialmente, elegimos $M = D$ y $N = L + U$:

$$x^{(k+1)} = D^{-1}(L + U)x^{(k)} + D^{-1}b$$

A nivel de componentes, la fórmula para cada incógnita i es:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij} x_j^{(k)} \right)$$

2.2 Método de Gauss-Seidel

El método de Gauss-Seidel mejora la velocidad al utilizar los nuevos valores $x_j^{(k+1)}$ en cuanto se calculan dentro de la misma iteración. Elegimos $M = D - L$ y $N = U$:

$$(D - L)x^{(k+1)} = Ux^{(k)} + b \implies x^{(k+1)} = (D - L)^{-1}Ux^{(k)} + (D - L)^{-1}b$$

A nivel de componentes, la fórmula es:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j < i} a_{ij} x_j^{(k+1)} - \sum_{j > i} a_{ij} x_j^{(k)} \right)$$

3. Análisis de Convergencia

Un método iterativo converge a la solución exacta para cualquier vector inicial $x^{(0)}$ si y solo si el **radio espectral** de la matriz de iteración B es estrictamente menor que 1:

$$\rho(B) < 1$$

donde el radio espectral $\rho(B)$ es el máximo valor absoluto de los autovalores de B :

$$\rho(B) = \max\{|\lambda_i| : \lambda_i \text{ es autovalor de } B\}$$

Condiciones Suficientes de Convergencia

1. **Matrices Diagonalmente Dominantes por Filas:** Si en cada fila de A , el elemento de la diagonal es mayor en valor absoluto que la suma de los demás elementos de la fila:

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}| \quad \forall i$$

Tanto Jacobi como Gauss-Seidel **tienen garantizada la convergencia**.

2. **Matrices Simétricas y Definidas Positivas:** El método de Gauss-Seidel converge siempre si la matriz es simétrica y definida positiva.

4. Cálculo Aproximado de Autovalores: Método de las Potencias

Para matrices gigantescas, calcular analíticamente las raíces del polinomio característico es imposible. El **Método de las Potencias** es un algoritmo iterativo que aproxima el **autovalor dominante** (el de mayor valor absoluto λ_1) y su autovector asociado v_1 .

Dado un vector inicial $y^{(0)}$, realizamos la recurrencia:

$$y^{(k+1)} = A \cdot x^{(k)}$$

$$x^{(k+1)} = \frac{y^{(k+1)}}{\|y^{(k+1)}\|} \quad (\text{normalización})$$

El cociente de Rayleigh aproxima el autovalor dominante en cada etapa:

$$\lambda_1^{(k)} \approx \frac{(x^{(k)})^T A x^{(k)}}{(x^{(k)})^T x^{(k)}}$$

5. El Toque Informático

A continuación, implementamos en Matlab/Octave los métodos de Jacobi y Gauss-Seidel para resolver el sistema diagonalmente dominante:

$$\begin{pmatrix} 4 & 1 & 1 \\ 1 & 5 & 2 \\ 1 & 2 & 5 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 6 \\ 8 \\ 8 \end{pmatrix}$$

Su solución exacta es $x = [1; 1; 1]$.

```
% Definición de la matriz A y el vector b
A = [4, 1, 1;
     1, 5, 2;
     1, 2, 5];
b = [6; 8; 8];

n = size(A, 1);
x0 = zeros(n, 1); % Estimación inicial
tol = 1e-6;
max_iter = 100;

% 1. Implementación del Método de Jacobi
function [x, it] = jacobi_iterativo(A, b, x0, tol, max_iter)
    n = length(b);
    x = x0;
    x_prev = x0;
    D = diag(diag(A));
    LU = A - D; % L + U con signo cambiado

    for it = 1:max_iter
        % Ecuación matricial de Jacobi: x^(k+1) = D^-1 * (-(L+U)*x^(k) + b)
        x = D \ (-LU * x_prev + b);

        if norm(x - x_prev, inf) < tol
            return;
        end
        x_prev = x;
    end
end
```

```

end

% 2. Implementación del Método de Gauss-Seidel
function [x, it] = gauss_seidel_iterativo(A, b, x0, tol, max_iter)
    n = length(b);
    x = x0;
    x_prev = x0;

    for it = 1:max_iter
        % Nivel de componentes para aprovechar valores actualizados en el momento
        for i = 1:n
            suma = b(i);
            for j = 1:n
                if j != i
                    suma = suma - A(i, j) * x(j);
                end
            end
            x(i) = suma / A(i, i);
        end

        if norm(x - x_prev, inf) < tol
            return;
        end
        x_prev = x;
    end
end

% Ejecución y comparación
[x_jac, it_jac] = jacobi_iterativo(A, b, x0, tol, max_iter);
[x_gs, it_gs] = gauss_seidel_iterativo(A, b, x0, tol, max_iter);

printf("Resultados de los Métodos Iterativos:\n");
printf("Jacobi:      Iteraciones: %d | Solución: [%s]\n", it_jac, sprintf(" %.4f", x_jac));
printf("Gauss-Seidel: Iteraciones: %d | Solución: [%s]\n", it_gs, sprintf(" %.4f", x_gs));

```

Gauss-Seidel requiere aproximadamente la mitad de iteraciones que Jacobi gracias a la retroalimentación inmediata de las incógnitas calculadas.

6. Ejercicios Resueltos

Ejercicio 1

Determinar si se garantiza la convergencia de los métodos de Jacobi y Gauss-Seidel para resolver el siguiente sistema:

$$\begin{cases} 5x - y + 2z = 6 \\ x + 4y - z = 4 \\ 2x + y - 6z = -3 \end{cases}$$

Solución analizando la Dominancia Diagonal:

1. Escribir la matriz de coeficientes A :

$$A = \begin{pmatrix} 5 & -1 & 2 \\ 1 & 4 & -1 \\ 2 & 1 & -6 \end{pmatrix}$$

2. Comprobar la condición de dominancia diagonal estricta por filas:

- **Fila 1:** $|a_{11}| = |5| = 5$. Suma de los demás elementos de la fila: $|a_{12}| + |a_{13}| = |-1| + |2| = 1 + 2 = 3$. Como $5 > 3$, se cumple para la fila 1.
- **Fila 2:** $|a_{22}| = |4| = 4$. Suma de los demás elementos: $|a_{21}| + |a_{23}| = |1| + |-1| = 1 + 1 = 2$. Como $4 > 2$, se cumple para la fila 2.
- **Fila 3:** $|a_{33}| = |-6| = 6$. Suma de los demás elementos: $|a_{31}| + |a_{32}| = |2| + |1| = 2 + 1 = 3$. Como $6 > 3$, se cumple para la fila 3.

3. **Conclusión:** Dado que la matriz A es estrictamente diagonalmente dominante por filas, la teoría matemática **garantiza de forma absoluta la convergencia** de los métodos de Jacobi y Gauss-Seidel para cualquier vector de partida inicial $x^{(0)}$.

7. Ejercicios Propuestos

1. Dada la matriz de iteración de Jacobi:

$$B = \begin{pmatrix} 0 & 0.5 \\ 0.5 & 0 \end{pmatrix}$$

Calcular sus autovalores y determinar el radio espectral $\rho(B)$. ¿Garantiza esto la convergencia del método?

2. Escribir analíticamente la primera iteración completa ($x^{(1)}$) del método de Jacobi y del método de Gauss-Seidel partiendo de $x^{(0)} = (0, 0, 0)$ para el sistema:

$$\begin{cases} 3x + y = 4 \\ x + 4y = 5 \end{cases}$$

3. Explicar por qué los métodos iterativos resultan sumamente ventajosos para matrices de adyacencia de la Web (algoritmo PageRank) frente a la eliminación de Gauss directa. *(Pista: Considera la cantidad de elementos nulos y la conservación de la dispersión de la matriz).*

Tema 9: Laboratorio de Programación Matemática

En el ámbito científico e industrial de la computación, el desarrollo rápido de algoritmos lineales y numéricos se realiza con lenguajes de alto nivel orientados a matrices como **Matlab** o **GNU Octave**. Este laboratorio enseña la sintaxis fundamental, el concepto crítico de la **vectorización de código** (esencial para maximizar la velocidad en CPU/GPU) y la resolución por software de los bloques algebraicos y numéricos estudiados previamente.

1. Sintaxis Básica de Matlab/Octave

Matlab y Octave tratan a cada variable como una matriz (por defecto de coma flotante de doble precisión).

1.1 Creación de Matrices y Vectores

- **Vectores:**

```
v_fila = [1, 2, 3];  
v_columna = [1; 2; 3];  
v_rango = 1:0.5:3; % Genera [1.0, 1.5, 2.0, 2.5, 3.0]  
v_lin = linspace(0, 1, 5); % Genera 5 puntos entre 0 y 1
```

- **Matrices:**

```
A = [1, 2; 3, 4];  
I = eye(3);      % Matriz Identidad de 3x3  
Z = zeros(2, 3); % Matriz de ceros de 2x3  
O = ones(4, 1);  % Vector de unos de 4x1  
R = rand(3, 3);  % Matriz aleatoria uniforme de 3x3
```

1.2 Indexación y Slicing (Selección de Rangos)

La indexación en Matlab/Octave es **1-indexed** (comienza en 1, no en 0):

```
A(1, 2)    % Elemento de la fila 1, columna 2
A(2, :)    % Fila 2 completa
A(:, 3)    % Columna 3 completa
A(1:2, 2:3) % Submatriz formada por las filas 1-2 y columnas 2-3
```

2. Operaciones Elemento a Elemento vs Operaciones Matriciales

Un error común al programar en Matlab/Octave es confundir las operaciones algebraicas de matrices con las operaciones aritméticas elemento a elemento.

- **Multiplicación Matricial Algebraica (*)**: Realiza el producto interno de filas por columnas. Requiere que las dimensiones coincidan ($m \times p$ con $p \times n$).

```
C = A * B;
```

- **Multiplicación Elemento a Elemento (.*)**: Multiplica cada componente de A por la correspondiente de B . Ambas deben tener el mismo tamaño exacto.

```
C = A .* B;
```

- **División y Potenciación Elemento a Elemento (./, .^)**:

```
C = A ./ B; % Cada elemento A(i,j) dividido por B(i,j)
D = A.^ 2; % Eleva cada componente de A al cuadrado
```

3. Vectorización de Código

La **vectorización** consiste en reescribir un algoritmo iterativo basado en bucles (**for** o **while**) mediante operaciones matriciales vectoriales compactas.

Por qué es crucial la vectorización

Octave y Matlab son lenguajes interpretados. Ejecutar un bucle **for** con millones de iteraciones introduce un gran coste de interpretación paso a paso en la CPU. Al utilizar expresiones vectorizadas, las operaciones se delegan a librerías compiladas de bajo nivel (BLAS y LAPACK) escritas en C/C++ y Fortran, las cuales están altamente optimizadas a nivel de registros de procesador y aprovechan instrucciones de hardware de tipo **SIMD** (Single Instruction, Multiple Data) y paralelismo multinúcleo.

4. El Toque Informático: Simulación y Benchmarking de Vectorización

A continuación, implementamos una prueba de rendimiento (benchmark) para contrastar el coste temporal de procesar una señal matemática (evaluar $y_i = \sin(x_i) \cdot x_i^2$) mediante un bucle **for** tradicional frente al enfoque vectorizado.

```
% Número de puntos de la simulación (1 millón de elementos)
N = 1000000;
x = linspace(-10, 10, N);

% 1. Enfoque No Vectorizado (Bucle For)
tic; % Iniciar cronómetro
y_for = zeros(1, N);
for i = 1:N
    y_for(i) = sin(x(i)) * (x(i)^2);
end
tiempo_for = toc; % Detener cronómetro

% 2. Enfoque Vectorizado (Operaciones con el operador punto '.')
tic;
y_vec = sin(x) .* (x.^ 2);
tiempo_vec = toc;

printf("Estudio de Rendimiento (Evaluación de %d elementos):\n", N);
printf("Tiempo con Bucle For: %.6f segundos\n", tiempo_for);
```

```
printf("Tiempo Vectorizado: %.6f segundos\n", tiempo_vec);
printf("Factor de aceleración (Speedup): %.1f x\n", tiempo_for / tiempo_vec);

% Comprobación de que ambos métodos dan el mismo resultado exacto
max_diff = max(abs(y_for - y_vec));
printf("Diferencia máxima entre resultados: %.2e\n", max_diff);
```

La diferencia de rendimiento suele superar las **50 veces de aceleración** a favor de la vectorización, ilustrando por qué los informáticos deben evitar los bucles anidados en computación científica.

5. Ejercicios Resueltos

Ejercicio 1

Escribir un script de Matlab/Octave que:

1. Genere una matriz aleatoria cuadrada A de tamaño 4×4 .
2. Calcule su determinante. Si es invertible, resuelva el sistema $Ax = b$ para $b = [1; 2; 3; 4]$ usando el operador backslash.
3. Calcule la descomposición LU de la matriz y verifique que $P \cdot A - L \cdot U$ es nulo.

Solución (Script de Octave):

```
% 1. Generar la matriz aleatoria
A = rand(4, 4);
b = [1; 2; 3; 4];

% 2. Evaluar determinante y resolver
det_A = det(A);
printf("Determinante de A: %.6f\n", det_A);

if abs(det_A) > 1e-9
    x = A \ b;
    printf("El sistema es compatible determinado. Solución x:\n");
    disp(x);
else
    printf("La matriz es singular. No se puede resolver de forma única.\n");
end

% 3. Descomposición LU y verificación
[L, U, P] = lu(A);
verificacion = P * A - L * U;
error_max = max(max(abs(verificacion)));
```

```
printf("Error máximo en la reconstrucción P*A - L*U: %.2e\n", error_max);
```

6. Ejercicios Propuestos

1. Crear una matriz M de tamaño 5×5 con números enteros del 1 al 25 de forma compacta (usando rangos y la función `reshape`). Seleccionar la submatriz central de tamaño 3×3 .
2. Implementar una función de Octave que reciba una matriz A y verifique si es simétrica ($A = A^T$) y definida positiva (comprobando que todos sus autovalores sean estrictamente positivos).
3. Vectorizar el siguiente fragmento de código ineficiente:

```
n = 10000;  
A = rand(n, 1);  
B = rand(n, 1);  
resultado = 0;  
for i = 1:n  
    resultado = resultado + A(i) * B(i);  
end
```

(Pista: Considera el producto escalar o la multiplicación de matrices de vectores traspuestos).

Glosario de Términos

- **Autovalor (Valor propio):** Escalar λ que satisface la ecuación característica $Av = \lambda v$ para algún autovector no nulo v . Representa el factor de escala de la transformación en dicha dirección.
- **Autovector (Vector propio):** Vector no nulo v que, bajo la multiplicación por una matriz A , mantiene su dirección original en el espacio, siendo únicamente escalado por el autovalor correspondiente.
- **Base:** Conjunto de vectores linealmente independientes que generan un espacio vectorial. Define el sistema de referencia de coordenadas para dicho espacio.
- **Cambio de Base:** Transformación semejante que relaciona las matrices asociadas a una aplicación lineal bajo dos bases de coordenadas distintas.
- **Característica de un Cuerpo:** Menor entero positivo p tal que la suma iterada del elemento neutro del producto (1) da como resultado el neutro de la suma (0). En cuerpos finitos, la característica siempre es un primo.
- **Coordenadas Homogéneas:** Representación de vectores en el plano o espacio de coordenadas proyectivas que añade un componente escalar ficticio para unificar transformaciones lineales (rotación) y afines (traslación) en una única matriz.
- **Cuerpo de Galois:** Estructura algebraica de cuerpo con un número finito de elementos de orden p^m , donde p es primo y $m \geq 1$.
- **Descomposición LU:** Factorización de una matriz cuadrada en el producto de una triangular inferior con unos en la diagonal (L) y una triangular superior (U).
- **Descomposición de Cholesky:** Factorización alternativa y optimizada de una matriz simétrica y definida positiva en el producto de una triangular inferior por su traspuesta ($L \cdot L^T$).
- **Dimensión:** Cardinal o número de elementos de cualquiera de las bases que componen un espacio vectorial de dimensión finita.
- **Espacio Euclídeo:** Espacio vectorial real dotado de un producto escalar que define nociones geométricas de longitud (norma), ángulo y distancia.
- **Inverso Modular:** Elemento x tal que $a \cdot x \equiv 1 \pmod{m}$. Su existencia está condicionada a que a y m sean coprimos ($\text{extmcd}(a, m) = 1$).
- **Matriz de Permutación:** Matriz identidad cuyas filas se han intercambiado para modelar formalmente los intercambios de filas requeridos en las estrategias de pivotaje parcial de Gauss.

- **Nodos de Chebyshev:** Distribución óptima no equiespaciada de puntos en un intervalo que reduce a la mínima expresión el error de interpolación polinómica y evita el fenómeno de Runge.
- **Radio Espectral:** Valor absoluto máximo del conjunto de autovalores de una matriz. En métodos iterativos, determina la convergencia si su valor es estrictamente menor a 1.
- **Vectorización:** Técnica de desarrollo de software consistente en formular algoritmos iterativos mediante multiplicación y operaciones matriciales compactas, aprovechando instrucciones SIMD de bajo nivel del procesador.

Bibliografía Recomendada

1. **Apostol, T. M. (1969).** *Calculus*, Volumen 2: Cálculo multivariable y Álgebra lineal con aplicaciones. Editorial Reverté.
 - *Nota:* Rigurosa aproximación que vincula formalmente los espacios vectoriales y las aplicaciones lineales con su interpretación geométrica y física.
2. **Burden, R. L., & Faires, J. D. (2011).** *Análisis Numérico* (9ª ed.). Cengage Learning.
 - *Nota:* Texto de cabecera indispensable para el Bloque II, especialmente para comprender la convergencia de Jacobi, Gauss-Seidel y las factorizaciones LU y Cholesky.
3. **Strang, G. (2016).** *Introduction to Linear Algebra* (5th ed.). Wellesley-Cambridge Press.
 - *Nota:* Una obra maestra del MIT, muy recomendada para entender conceptualmente la ortogonalidad, los espacios vectoriales y la descomposición en valores propios.
4. **Golub, G. H., & Van Loan, C. F. (2013).** *Matrix Computations* (4th ed.). Johns Hopkins University Press.
 - *Nota:* La biblia definitiva del coste computacional y las estrategias de hardware (BLAS/LAPACK) para factorizaciones QR, LU y Cholesky.
5. **Meyer, C. D. (2000).** *Matrix Analysis and Applied Linear Algebra*. SIAM.
 - *Nota:* Excelentes conexiones conceptuales y ejercicios de aplicación directa a la computación en sistemas de información y grafos.